

Step 4 - Window Functions & Tuning

Phase A · Running Balance · Rolling Spend · Broadcast · ~20 min

Goal: The highest-yield interview material: window functions (running balance, 30-day rolling spend, top-N, dedup-keep-latest), native-vs-UDF masking, and broadcast joins with plan reading.
Artifact: 04_windows_tuning.ipynb

LEARN

1. Window functions = aggregates that keep every row. `groupBy` collapses 10,000 rows to 15 totals; a window adds the aggregate as a column on each row: 'this transaction AND the running balance after it.'

```
Window.partitionBy("account_id")      # restart per account
  .orderBy("transaction_timestamp")    # define sequence
  .rowsBetween(start, end)             # the frame
```

2. `rowsBetween` counts rows; `rangeBetween` measures values (cast timestamp to long = seconds). 30-day windows need `rangeBetween` - a classic interview trap.

3. UDFs are the last resort. A Python UDF is opaque to the Catalyst optimizer; native functions are optimized. Try native first, always.

4. Broadcast joins kill shuffles. Ship the 15-row branch table to every worker instead of shuffling 10,000 transactions.

LAB — notebook 04_windows_tuning (Serverless)

Cell 1: reload `df_txn` (explicit schema), `df_customer`, `df_branch`. Add:

```
from pyspark.sql.window import Window
```

Cell 2 — Running balance per account:

```
df_signed = df_txn.withColumn("signed_amount",
    F.when(F.col("transaction_type") == "PAYROLL_DEPOSIT", F.col("amount_cad"))
    .otherwise(-F.col("amount_cad")))

w_running = Window.partitionBy("account_id") \
    .orderBy("transaction_timestamp") \
    .rowsBetween(Window.unboundedPreceding, Window.currentRow)

df_balance = df_signed.withColumn("running_balance",
    F.round(F.sum("signed_amount").over(w_running), 2))

df_balance.filter(F.col("account_id") == df_balance.select("account_id").first()[0]) \
    .select("transaction_timestamp", "transaction_type", "signed_amount", "running_balance") \
    .orderBy("transaction_timestamp").show(10, truncate=False)
```

Cell 3 — 30-day rolling spend (time-based frame):

```
days = lambda n: n * 86400
w_30d = Window.partitionBy("customer_id") \
    .orderBy(F.col("transaction_timestamp").cast("long")) \
    .rangeBetween(-days(30), 0)

df_rolling = df_txn \
    .withColumn("spend_30d", F.round(F.sum("amount_cad").over(w_30d), 2)) \
    .withColumn("txn_count_30d", F.count("*").over(w_30d))
```

Cell 4 — Top-3 customers per branch:

```
df_cust_branch = df_txn.groupBy("branch_id", "customer_id") \
    .agg(F.round(F.sum("amount_cad"), 2).alias("total_spend"))

w_rank = Window.partitionBy("branch_id").orderBy(F.col("total_spend").desc())
df_top3 = df_cust_branch.withColumn("rank_in_branch", F.row_number().over(w_rank)) \
    .filter(F.col("rank_in_branch") <= 3)
```

Know the trio: row_number (1,2,3,4 - no ties), rank (1,2,2,4 - ties skip), dense_rank (1,2,2,3 - no skip). Asked constantly.

Cell 5 — Dedup-keep-latest (the production pattern):

```
dupe = df_txn.limit(1) \
    .withColumn("transaction_timestamp",
        F.col("transaction_timestamp") + F.expr("INTERVAL 2 HOURS")) \
    .withColumn("amount_cad", F.lit(9999.99))
df_with_dupes = df_txn.union(dupe) # 10,001

w_dedup = Window.partitionBy("transaction_id") \
    .orderBy(F.col("transaction_timestamp").desc())
df_deduped = df_with_dupes.withColumn("rn", F.row_number().over(w_dedup)) \
    .filter(F.col("rn") == 1).drop("rn") # 10,000 - kept the LATEST
```

Beats dropDuplicates([key]) which keeps an ARBITRARY row; this keeps the latest, deterministically - what banks require.

Cell 6 — Native vs UDF masking:

```
# Native (Catalyst-optimized) - preferred:
df_native = df_customer.withColumn("sin_masked",
    F.concat(F.lit("***-***-"), F.substring(F.col("sin"), 8, 3)))

# UDF (works, but opaque to the optimizer):
from pyspark.sql.types import StringType as STt
@F.udf(returnType=STt())
def mask_sin_udf(sin):
    return f"***-***-{sin[-3:]}" if sin else "***-***-***"
df_udf = df_customer.withColumn("sin_masked", mask_sin_udf(F.col("sin")))
```

Cell 7 — Broadcast join + read the plan:

```
df_joined = df_txn.join(F.broadcast(df_branch), on="branch_id", how="left")
df_joined.explain() # find BroadcastHashJoin (vs shuffle-heavy SortMergeJoin)
```

COMMIT + CHECKPOINT

Export to pyspark/. Pass criteria: running balance accumulates per account; dedup 10,001 to 10,000; BroadcastHashJoin visible in the plan. Phase A complete - four progressively harder notebooks visible on GitHub.